

# Deep Learning II

## Language Modeling, Transformer, GPT

Chengwei QIN

AI Thrust, Information Hub

2025.11.20

- 1 **Language Modeling: the Next-Token Prediction Paradigm**
  - Conditional distribution estimation
- 2 **The Transformer Architecture**
  - Core components: attention, FFN, positional encoding
  - Layer structure: encoder vs. decoder
- 3 **Generative Pre-trained Transformer (GPT) Models**
  - 3-stage pipeline: pre-training, fine-tuning, reinforcement learning
- 4 **The Scaling Law & Future Challenges**

# Language Modeling: the Next-Token Prediction Paradigm

# What is Language Modeling?

- **Language Modeling (The Task):** The task of capturing the statistical properties and patterns of human language.
- At its core, it's about answering the question: "What comes next?"

## Example: Predicting the next word

Given the context: "The students opened their..." A system must assign probabilities:

- books (high probability)
- laptops (medium probability)
- bananas (very low probability)

# What is Language Modeling?

- **Language Modeling (The Task):** The task of capturing the statistical properties and patterns of human language.
- At its core, it's about answering the question: "What comes next?"

## Example: Predicting the next word

Given the context: "The students opened their..." A system must assign probabilities:

- books (high probability)
  - laptops (medium probability)
  - bananas (very low probability)
- 
- **Language Model (The System):** A Language Model (LM) is the system, model, or algorithm that *performs* this task.
  - To do this, the system must learn a **probability distribution** over all possible sequences.

# What is a Language Model (LM)? [Formal]

- **Tokens ( $\mathcal{V}$ ):** The basic building blocks (symbols, words, or subwords) of LMs.
- **Language Model ( $P$ ):** A probability distribution over all possible sequences of tokens ( $\mathcal{V}^*$ ).

# What is a Language Model (LM)? [Formal]

- **Tokens ( $\mathcal{Y}$ ):** The basic building blocks (symbols, words, or subwords) of LMs.
- **Language Model ( $P$ ):** A probability distribution over all possible sequences of tokens ( $\mathcal{Y}^*$ ).
- **Core Task:** The model must compute the conditional probability of the **next** token  $y_t$  given the preceding history  $y^{t-1}$ .

$$P(y_t|y^{t-1})$$

- **Sequence Probability:** The probability of an entire sentence is the product of these conditional probabilities:

$$P(y^T) = \prod_{t=1}^T P(y_t|y^{t-1})$$

# How is an LM Learned? (MLE)

- 1 **Hypothesis Class ( $\mathcal{H}$ ):** We first choose an architecture (a set of models  $h_\theta$ ).  $h_\theta(y^{t-1})$  represents the model's predicted conditional distribution.
- 2 **Training Data ( $y^T$ ):** A large corpus of text.
- 3 **Learning Objective: Maximum Likelihood Estimation (MLE)**
  - Find the parameters  $\hat{\theta}$  that maximize the probability of observing the training data.

$$\hat{\theta} := \arg \max_{\theta \in \Theta} \prod_{t=1}^T h_\theta(y^{t-1})[y_t]$$

# How is an LM Learned? (MLE)

- ➊ **Hypothesis Class ( $\mathcal{H}$ ):** We first choose an architecture (a set of models  $h_\theta$ ).  $h_\theta(y^{t-1})$  represents the model's predicted conditional distribution.
- ➋ **Training Data ( $y^T$ ):** A large corpus of text.
- ➌ **Learning Objective: Maximum Likelihood Estimation (MLE)**
  - Find the parameters  $\hat{\theta}$  that maximize the probability of observing the training data.

$$\hat{\theta} := \arg \max_{\theta \in \Theta} \prod_{t=1}^T h_\theta(y^{t-1})[y_t]$$

- ➍ **In Practice (Log Loss):** This is equivalent to minimizing the Negative Log-Likelihood (Logarithmic Loss).

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \sum_{t=1}^T -\log h_\theta(y^{t-1})[y_t]$$

- ➎ This loss is typically minimized using gradient-based algorithms (like SGD, AdamW).

# How is an LM Learned? (PyTorch, Part 1: Setup)

## Conceptual PyTorch Setup

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 # 1. Hypothesis Class (e.g., a Transformer)
5 # model = MyTransformerModel(vocab_size, ...)
6 # h_theta
7 model = ... # (Instantiate our model)
8
9 # 2. Training Data
10 # context = "The students opened" -> [5, 27, 89]
11 # target = "their" -> [101]
12 context_ids = torch.tensor([[5, 27, 89]]) #  $y^{(t-1)}$ 
13 target_ids = torch.tensor([101]) #  $y_t$ 
14
15 # 3. Learning Objective (Negative Log-Likelihood)
16 # nn.CrossEntropyLoss is the key!
17 # It directly implements:  $-\log h_{\theta}(y^{(t-1)})[y_t]$ 
18 loss_function = nn.CrossEntropyLoss()
19 # 4. Optimization Algorithm (e.g., Adam)
20 optimizer = optim.Adam(model.parameters(), lr=0.001)
```

# How is an LM Learned? (PyTorch, Part 2: Training Step)

## Conceptual PyTorch Training Step

```
1  # --- A Single Training Step ---
2  # Zero out gradients from last step
3  optimizer.zero_grad()
4
5  # Forward pass: Get predictions (logits)
6  # Logits =  $h_{\theta}(y^{(t-1)})$ 
7  logits = model(context_ids)
8  # E.g., logits shape: [1, vocab_size]
9
10 # Compute the Log Loss
11 # This is the core of MLE
12 loss = loss_function(logits, target_ids)
13
14 # Backward pass: Compute gradients
15 loss.backward()
16 # Update model parameters ( $\theta$ )
17 optimizer.step()
```

# Evolution of Architectures

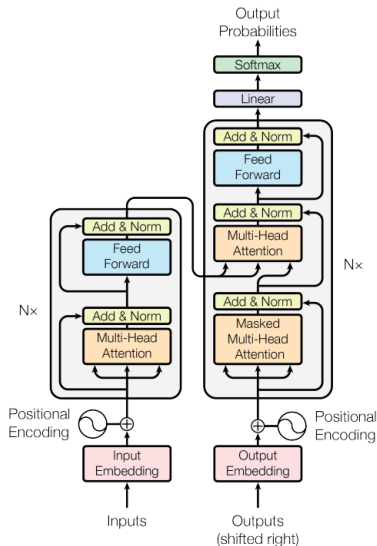
- The key to learning is choosing a good architecture for the hypothesis class  $\mathcal{H}$ .
- Major architectures include:
  - **n-gram Models:** Use a fixed-length context of  $n - 1$  words.
  - **Recurrent Neural Networks (RNNs):** Process sequences word-by-word, capturing temporal dependencies.
  - **Long Short-Term Memory (LSTMs):** A type of RNN designed to better capture long-range dependencies.
  - **Transformer Models:** Use self-attention to process sequences in parallel (e.g., BERT, GPT).

# The Transformer Architecture

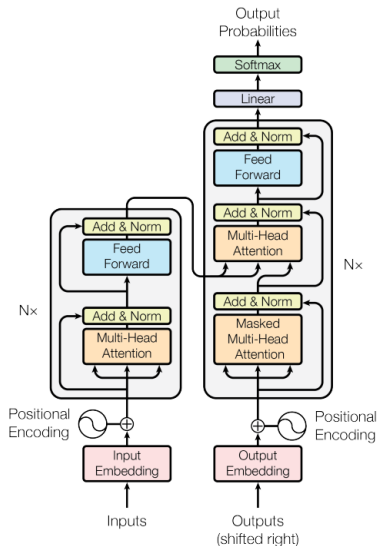
# Transformer Architecture Overview

## Encoder Flow (Left Stack):

- 1 **Input:** a sequence of tokens (through tokenization)
- 2 **Process:** Embedding + PE
- 3 **N x Layers:**
  - Multi-Head Attention
  - Feed Forward
- 4 **Output:** Contextual representation (fed to Decoder)



# Transformer Architecture Overview



## Encoder Flow (Left Stack):

- 1 **Input:** a sequence of tokens (through tokenization)
- 2 **Process:** Embedding + PE
- 3  **$N \times$  Layers:**
  - Multi-Head Attention
  - Feed Forward
- 4 **Output:** Contextual representation (fed to Decoder)

## Decoder Flow (Right Stack):

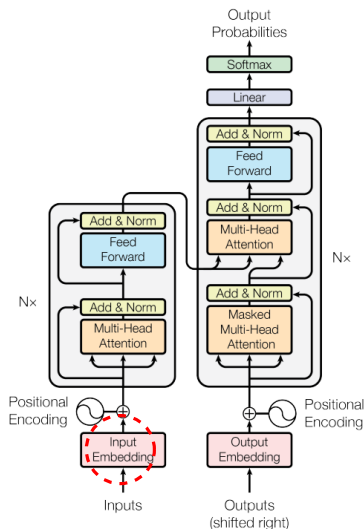
- 1 **Input:** Outputs (shifted right)
- 2 **Process:** Embedding + PE
- 3  **$N \times$  Layers:**
  - **Masked** MHA
  - Multi-Head Attention (**Cross-Attention**)
  - Feed Forward
- 4 **Output:** Linear + Softmax for "Output Probabilities"

# Component 1: Input Embedding

- **Vocabulary ( $\mathcal{Y}$ ):** The set of all possible tokens.
  - e.g., In GPT-3,  $|\mathcal{Y}| \sim 50,000$ .
- **Embedding Matrix ( $M$ ):** A large, **learnable** matrix  $M \in \mathbb{R}^{d \times |\mathcal{Y}|}$ .
  - $d$  is the embedding dimension (e.g.,  $d = 12,288$  for GPT-3).
- **Mapping:** Each token  $y$  (represented as a one-hot vector  $e_y$ ) is mapped to a dense vector  $x_y$ .

$$x_y := M e_y \in \mathbb{R}^d$$

- This is the **first learnable** layer of the network, converting discrete tokens into continuous vectors.



## Component 2: Positional Encoding

- **Problem:** Transformer blocks (like MHA and FFN) are **permutation invariant**.
- This means they cannot capture word order, which is crucial for language modeling.

## Component 2: Positional Encoding

- **Problem:** Transformer blocks (like MHA and FFN) are **permutation invariant**.
- This means they cannot capture word order, which is crucial for language modeling.
- **Solution:** Positional embeddings are introduced.
  - A vector  $p_i \in \mathbb{R}^d$  is defined for each position  $i$ .
  - This  $P$  matrix is **added** to the input token embeddings  $X$ .

$$X' = X + P \in \mathbb{R}^{t \times d}, t = \text{seq\_len}$$

- **Implementation:**  $P$  can be learned or predefined (e.g., using Rotary Position Embedding (RoPE)).
  - **RoPE:** modifies the Query ( $Q$ ) and Key ( $K$ ) vectors **inside** the attention mechanism based on their relative positions.

# PyTorch Code: Token Embedding ('nn.Embedding')

## Conceptual PyTorch Code

```
1 import torch
2 import torch.nn as nn
3 import math
4 # --- Model Hyperparameters ---
5 vocab_size = 50000 # |Y|
6 d_model = 768 # d (embedding dimension)
7 max_seq_len = 1024 # Maximum sequence length
8 # --- Token Embedding Layer ---
9 # This is our learnable matrix M
10 # It's a simple lookup table: index -> vector
11 token_embedding_layer = nn.Embedding(vocab_size, d_model)
12 # --- Example Usage ---
13 # Batch of 2 sentences, 5 tokens each
14 # (e.g., [ [ "The", "cat", "sat", "on", "the" ],
15 #          [ "A", "dog", "ran", ".", "<pad>" ] ])
16 input_ids = torch.randint(0, vocab_size, (2, 5))
17 # input_ids.shape: [batch_size=2, seq_len=5]
18 # Pass token IDs through the embedding layer
19 #  $X = M * e_y$ 
20 token_embeddings = token_embedding_layer(input_ids)
21 # token_embeddings.shape: [2, 5, d_model=768]
```

# PyTorch Code: RoPE (1/3 - Pre-computing Frequencies)

## RoPE '.\_\_init\_\_': Pre-computing $\theta_i$ values

```
1 class RoPE(nn.Module):
2     def __init__(self, dim, max_seq_len, base=10000):
3         super().__init__()
4         self.dim = dim
5         self.max_seq_len = max_seq_len
6         self.base = base
7         # 1. Calculate frequencies (theta_i)
8         # theta_i = 1.0 / (base^(2i / dim))
9         theta = 1.0 / (self.base ** (
10             torch.arange(0, self.dim, 2).float() / self.dim
11         ))
12         # 2. Create position indices: [0, 1, ..., max_seq_len-1]
13         t = torch.arange(self.max_seq_len)
14         # 3. Calculate all m*theta_i combinations
15         # freqs shape: [max_seq_len, dim / 2]
16         freqs = torch.outer(t, theta)
17         # 4. Pre-compute cosine and sine values
18         # We store these as non-parameter buffers
19         self.register_buffer("freqs_cos", freqs.cos())
20         self.register_buffer("freqs_sin", freqs.sin())
```

# PyTorch Code: RoPE (2/3 - Applying the Rotation)

## RoPE 'apply\_rotary\_emb': The core rotation logic

```
1 def apply_rotary_emb(x, cos, sin):
2     # x.shape: [batch, seq_len, num_heads, head_dim]
3     # --- Split x into two halves (x1, x2) ---
4     # x1 = [..., 0, 2, 4, ...], x2 = [..., 1, 3, 5, ...]
5     x1 = x[..., ::2]
6     x2 = x[..., 1::2]
7     # --- Get the correct sin/cos for this seq_len ---
8     # cos/sin shape: [1, seq_len, 1, dim/2]
9     seq_len = x.shape[1]
10    cos = cos[:, :seq_len].unsqueeze(0).unsqueeze(2)
11    sin = sin[:, :seq_len].unsqueeze(0).unsqueeze(2)
12    # --- Apply the 2D rotation matrix ---
13    # [cos, -sin] [x1] = [x1*cos - x2*sin] (out1)
14    # [sin,  cos] [x2] = [x1*sin + x2*cos] (out2)
15    out1 = x1 * cos - x2 * sin
16    out2 = x1 * sin + x2 * cos
17    # --- Re-assemble the vector ---
18    # stack: [batch, seq_len, num_heads, dim/2, 2]
19    out = torch.stack([out1, out2], dim=-1)
20    # flatten: [batch, seq_len, num_heads, dim]
21    return out.flatten(dim=-2)
```

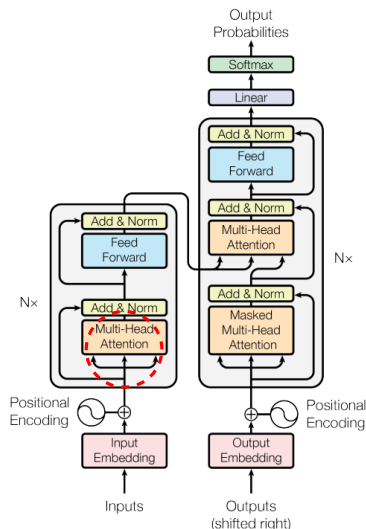
# PyTorch Code: RoPE (3/3 - Usage in Transformer Layer)

## Conceptual Transformer 'forward' pass (simplified)

```
1 class MyTransformerLayer(nn.Module):
2     def __init__(self, d_model, n_head):
3         # ... (other layers: self.wq, self.wk, self.wv, ...)
4         self.rop = RoPE(dim=d_model // n_head, max_seq_len=2048)
5     def forward(self, x):
6         # x.shape: [batch_size, seq_len, d_model]
7         # 1. Project to Q, K, V
8         # q, k, v shape: [batch, seq_len, n_head, head_dim]
9         q = self.wq(x).view(...)
10        k = self.wk(x).view(...)
11        v = self.wv(x).view(...)
12        # 2. !!! APPLY RoPE !!!
13        # We apply rotation *after* projection
14        q_rotated = apply_rotary_emb(q, self.rop.freqs_cos,
15                                     self.rop.freqs_sin)
16        k_rotated = apply_rotary_emb(k, self.rop.freqs_cos,
17                                     self.rop.freqs_sin)
18        # V is *never* rotated
19        # 3. Pass *rotated* Q/K to attention
20        # attention_output = self.attention(q_rotated, k_rotated, v)
21        ...
```

# Component 3: Multi-Head Attention (MHA)

- **What is MHA?** It is the core attention mechanism of the Transformer, replacing complex recurrence with parallel “self-attention”.
- **Why is it important?**
  - **Multi-Head:** It allows the model to jointly attend to information from different representation subspaces.
  - **Parallel:** It's highly parallelizable, making it efficient for modern hardware (GPUs/TPUs).
- **Core Idea (The “Heads”):**
  - The “Multi-Head” design is built from  $h$  (e.g., 12) identical, parallel units called “Heads”.
- **Transition:**
  - To understand the full MHA, we must first understand its core building block: the [Scaled Dot-Product Attention](#) (i.e., a “single head”).



# Scaled Dot-Product Attention (“Head”)

The core logic of a single attention head  $i$ .

- **Project:** Create Query, Key, and Value matrices from the input  $X \in \mathbb{R}^{t \times d}$  ( $t=\text{seq\_len}$ ,  $d=\text{model\_dim}$ ) (after position embedding) using learnable weights  $W_q^i, W_k^i, W_v^i \in \mathbb{R}^{d \times d'}$  ( $d'=\text{head\_dim} = d/h$ ).
  - $Q_i := XW_q^i \in [t, d']$
  - $K_i := XW_k^i \in [t, d']$
  - $V_i := XW_v^i \in [t, d']$

# Scaled Dot-Product Attention (“Head”)

The core logic of a single attention head  $i$ .

- **Project:** Create Query, Key, and Value matrices from the input  $X \in \mathbb{R}^{t \times d}$  ( $t=\text{seq\_len}$ ,  $d=\text{model\_dim}$ ) (after position embedding) using learnable weights  $W_q^i, W_k^i, W_v^i \in \mathbb{R}^{d \times d'}$  ( $d'=\text{head\_dim} = d/h$ ).
  - $Q_i := XW_q^i \in [t, d']$
  - $K_i := XW_k^i \in [t, d']$
  - $V_i := XW_v^i \in [t, d']$
- **Calculate Scores:** Compute scaled dot-product attention scores. “How much does each token ‘query’ match each ‘key’?”

$$S_i := \frac{(Q_i K_i^\top)}{\sqrt{d'}} \in \mathbb{R}^{t \times t}$$

# Scaled Dot-Product Attention (“Head”)

The core logic of a single attention head  $i$ .

- **Project:** Create Query, Key, and Value matrices from the input  $X \in \mathbb{R}^{t \times d}$  ( $t=\text{seq\_len}$ ,  $d=\text{model\_dim}$ ) (after position embedding) using learnable weights  $W_q^i, W_k^i, W_v^i \in \mathbb{R}^{d \times d'}$  ( $d'=\text{head\_dim} = d/h$ ).
  - $Q_i := XW_q^i \in [t, d']$
  - $K_i := XW_k^i \in [t, d']$
  - $V_i := XW_v^i \in [t, d']$
- **Calculate Scores:** Compute scaled dot-product attention scores. “How much does each token ‘query’ match each ‘key’?”

$$S_i := \frac{(Q_i K_i^\top)}{\sqrt{d'}} \in \mathbb{R}^{t \times t}$$

- **Get Weights (Softmax):** Apply softmax row-wise to get attention weights (probabilities).

$$P_i := \text{softmax}(S_i) \in \mathbb{R}^{t \times t}$$

- **Get Output:** Weighted sum of Values based on attention weights.

$$X_i := P_i V_i \in \mathbb{R}^{t \times d'}$$

# Why Scale the Dot Products? (The $1/\sqrt{d'}$ factor)

- **The Problem:** The dot product  $\mathbf{Q}_i \mathbf{K}_i^\top$  can grow very large in magnitude, especially if the dimension  $d'$  is large.
- **The Effect:** Large inputs (e.g., '[10, 80, 5, -100]') pushed into a 'softmax' function cause it to "saturate".
  - 'softmax([1, 2])'  $\rightarrow$  '[0.27, 0.73]' (Soft probabilities)
  - 'softmax([10, 80])'  $\rightarrow$  '[0.0, 1.0]' (Hard, one-hot probabilities)
- **The Gradient Issue:**
  - When 'softmax' saturates to 0 or 1, its gradient becomes extremely small (it "vanishes").
  - **Vanishing gradients** make training very slow or impossible.

# Why Scale the Dot Products? (The $1/\sqrt{d'}$ factor)

- **The Problem:** The dot product  $\mathbf{Q}_i \mathbf{K}_i^\top$  can grow very large in magnitude, especially if the dimension  $d'$  is large.
- **The Effect:** Large inputs (e.g., '[10, 80, 5, -100]') pushed into a 'softmax' function cause it to "saturate".
  - 'softmax([1, 2])'  $\rightarrow$  '[0.27, 0.73]' (Soft probabilities)
  - 'softmax([10, 80])'  $\rightarrow$  '[0.0, 1.0]' (Hard, one-hot probabilities)
- **The Gradient Issue:**
  - When 'softmax' saturates to 0 or 1, its gradient becomes extremely small (it "vanishes").
  - **Vanishing gradients** make training very slow or impossible.
- **The Solution:**
  - The authors of "Attention Is All You Need" assumed  $\mathbf{q}$  and  $\mathbf{k}$  have mean 0 and variance 1. This means the dot product  $\mathbf{Q}_i \mathbf{K}_i^\top$  has mean 0 but variance  $d'$ .
  - To counteract this, they divide by the standard deviation,  $\sqrt{d'}$ .
  - This **rescales the variance back to 1**, keeping the inputs to 'softmax' small and mitigating saturation.

# PyTorch Code: Scaled Dot-Product Attention

## Functional PyTorch Implementation

```
1 import torch.nn.functional as F
2 import math
3 def scaled_dot_product_attention(Q, K, V, mask=None):
4     # Q, K, V shape: [Batch, Num_Heads, Seq_Len, Head_Dim]
5     # 1. Get head dimension
6     d_k = Q.size(-1)
7     # 2. Calculate Scores: (B, H, T, T)
8     #     (B, H, T, D) @ (B, H, D, T) -> (B, H, T, T)
9     scores = torch.matmul(Q, K.transpose(-2, -1))
10    # 3. Scale: (The focus of the last slide)
11    scores = scores / math.sqrt(d_k)
12    # 4. (Optional) Apply Mask
13    #     (e.g., for padding or for decoder look-ahead)
14    if mask is not None:
15        # Fills -infinity where mask is 1
16        scores = scores.masked_fill(mask == 1, -1e9)
17    # 5. Get Weights (Softmax)
18    weights = F.softmax(scores, dim=-1)
19    # 6. Get Output (Weighted sum of V)
20    #     (B, H, T, T) @ (B, H, T, D) -> (B, H, T, D)
21    output = torch.matmul(weights, V)
22    return output, weights
```

# Practice: Creating a Padding Mask

## The Problem: Batched Inputs

- We process sentences in batches.
- Sentences have different lengths.
- We pad short sentences with a 'pad\_token' (e.g., ID 0) so all tensors in the batch are the same size.
- We must **not** let the model "attend" to these padding tokens.

## Your Task: Create the Mask

- Our 'scaled\_dot\_product\_attention' function fills where 'mask == 1'.
- This means we need a mask that is False (or 0) for **real** tokens and True (or 1) for **pad** tokens.
- It must also have the broadcastable shape: [B, 1, 1, T].

## Input Tensor

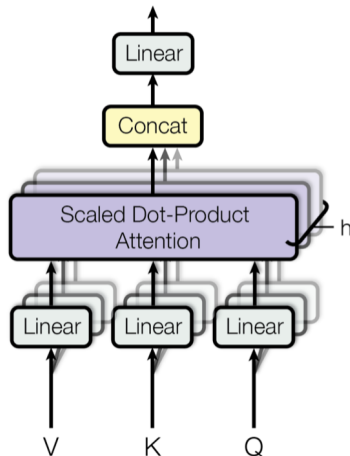
```
# Batch of 2 sentences, Seq length
# is 4, 0 is the <pad> token
input_ids = torch.tensor([
    [5, 8, 12, 0], # "The cat sat"
    [7, 2, 0, 0]  # "A dog"
])
# Shape: [B=2, T=4]
```

# Multi-Head Attention (MHA)

- **Recap:** SDPA (a single head  $i$ ) takes  $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i$  and returns one output  $\mathbf{X}_i$ .
- **MHA Logic:**
  - 1 Linearly project  $\mathbf{X}$  to get  $h$  different sets of  $(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$ .
  - 2 Run  $h$  independent SDPA heads in parallel.
  - 3 This yields  $h$  separate output matrices  $(\mathbf{X}_1, \dots, \mathbf{X}_h)$ .
  - 4 Concatenate all  $h$  outputs:  
 $\text{Concat}(\mathbf{X}_1, \dots, \mathbf{X}_h)$ .
  - 5 Apply a final linear projection  $\mathbf{W}^O$ .
- **Output Formula:**

$$\mathbf{X}_i = \text{SDPA}(\mathbf{X}\mathbf{W}_q^i, \mathbf{X}\mathbf{W}_k^i, \mathbf{X}\mathbf{W}_v^i)$$

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\mathbf{X}_1, \dots, \mathbf{X}_h)\mathbf{W}^O$$



**Figure:** MHA =  $h$  parallel SDPA heads.

# PyTorch Code: Multi-Head Attention (1/2: '\_\_init\_\_')

## Conceptual PyTorch 'nn.Module' (Simplified)

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, h):
3         super().__init__()
4         self.d_model = d_model # e.g., 768
5         self.h = h # e.g., 12
6         self.d_k = d_model // h # e.g., 64
7
8         # --- Learnable Weights ---
9
10        # Linear layers for Q, K, V (W_q, W_k, W_v)
11        # We define one big linear layer for each
12        self.q_linear = nn.Linear(d_model, d_model)
13        self.k_linear = nn.Linear(d_model, d_model)
14        self.v_linear = nn.Linear(d_model, d_model)
15
16        # Final output projection (W_o)
17        self.out_linear = nn.Linear(d_model, d_model)
18
19        # (forward method on next slide...)
```

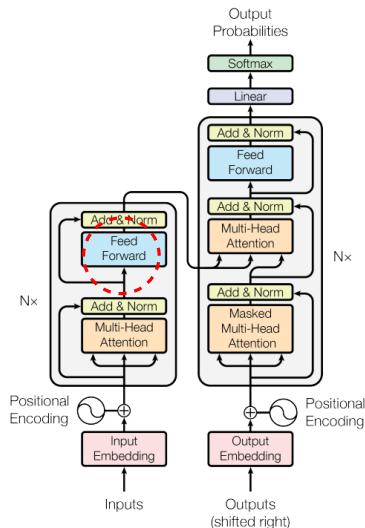
# PyTorch Code: Multi-Head Attention (2/2: 'forward')

## Conceptual PyTorch 'nn.Module' (Simplified)

```
1  def forward(self, x, mask=None):
2      # x shape: [Batch, Seq_Len (T), d_model]
3      batch_size = x.size(0)
4      # 1. Project and split into h heads
5      # Q/K/V shape: [B, T, d_model] -> [B, h, T, d_k]
6      Q = self.q_linear(x).view(batch_size, -1, self.h, self.d_k).
          transpose(1,2)
7      K = self.k_linear(x).view(batch_size, -1, self.h, self.d_k).
          transpose(1,2)
8      V = self.v_linear(x).view(batch_size, -1, self.h, self.d_k).
          transpose(1,2)
9
10     # 2. Run all heads in parallel (using our function)
11     # new_x shape: [B, h, T, d_k]
12     new_x, weights = scaled_dot_product_attention(Q, K, V, mask)
13
14     # 3. Concat and 4. Project
15     # [B, h, T, d_k] -> [B, T, h, d_k] -> [B, T, d_model]
16     new_x = new_x.transpose(1, 2).contiguous().view(batch_size,
          -1, self.d_model)
17     return self.out_linear(new_x)
```

# Component 4: Feed-Forward Layer (FFN)

- This block is applied **after** the MHA layer.
- It is a simple two-layer fully connected neural network.
- It is applied position-wise (i.e., independently to each token's vector).



# Component 4: Feed-Forward Layer (FFN)

- **Structure:**

- ① First layer expands dimension:  $X \rightarrow X'$  (using  $W_1, b_1$ ) with ReLU activation.

$$X' = \text{ReLU}(XW_1 + b_1)$$

- ② Second layer compresses back:  $X' \rightarrow X''$  (using  $W_2, b_2$ ).

$$X'' = X'W_2 + b_2$$

- The matrices  $W_1, W_2$  and vectors  $b_1, b_2$  are trainable parameters.
- (e.g., GPT-3 Small:  $d = 768 \rightarrow d_{ff} = 3072 \rightarrow d = 768$ ).

# PyTorch Code: Feed-Forward Layer (FFN)

## Part 1: '\_\_init\_\_'

```
1 import torch
2 import torch.nn as nn
3 class FeedForward(nn.Module):
4     # Implements FFN(X) = ReLU(XW1 + b1)W2 + b2
5     def __init__(self, d_model,
6                 d_ff):
7         super().__init__()
8         # W1 and b1 (e.g., 768 -> 3072)
9         self.linear_1 = nn.Linear(d_model, d_ff)
10
11     self.relu = nn.ReLU()
12     # W2 and b2 (e.g., 3072 -> 768)
13     self.linear_2 = nn.Linear(d_ff, d_model)
14
15 )
```

## Part 2: 'forward' & Usage

```
1 def forward(self, x):
2     # x shape: [Batch, Seq_Len, d_model]
3     # (1) x -> [B, T, d_ff]
4     x = self.relu(self.linear_1(x))
5     # (2) x -> [B, T, d_model]
6     x = self.linear_2(x)
7     return x
8
9 # --- Example Usage ---
10 d_model = 768
11 d_ff = 3072 # 768 * 4
12 ffn_block = FeedForward(d_model, d_ff)
13
14 # Create a dummy input from MHA
15 dummy_x = torch.randn(2, 10, d_model)
16 # [B, T, d_model]
17 output = ffn_block(dummy_x)
18 # output.shape: [2, 10, 768]
```

# The “Glue”: Add & Norm

- Each sub-layer (MHA, FFN) is wrapped by two operations:
- **Residual Connection (Add):**
  - The input to a layer is **added** to its output:  $X + \text{SubLayer}(X)$ .
  - This “shortcut” allows gradients to flow directly, preventing the vanishing gradient problem and enabling **very deep** networks.

# The “Glue”: Add & Norm

- Each sub-layer (MHA, FFN) is wrapped by two operations:
- **Residual Connection (Add):**
  - The input to a layer is **added** to its output:  $X + \text{SubLayer}(X)$ .
  - This “shortcut” allows gradients to flow directly, preventing the vanishing gradient problem and enabling **very deep** networks.
- **Layer Normalization (Norm):**
  - Normally, applied **before** the residual connection:  $X + \text{SubLayer}(\text{LN}(X))$  (although applied after the residual connection in the original design).
  - Stabilizes training by normalizing each token's vector  $\mathbf{x}_j$  (mean=0, var=1).
  - The mean  $\mu_j$  and variance  $\sigma_j^2$  are computed across the  $d$  dimensions:

$$\mu_j = \frac{1}{d} \sum_{k=1}^d \mathbf{x}_j[k], \sigma_j^2 = \frac{1}{d} \sum_{k=1}^d (\mathbf{x}_j[k] - \mu_j)^2$$

- Then applies **learnable**  $\gamma$  (scale) and  $\beta$  (shift) parameters.

$$\mathbf{x}'_j = \gamma \odot \left( \frac{\mathbf{x}_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \right) + \beta$$

# PyTorch Code: Add & Norm (Pre-Norm)

## Part 1: '\_\_init\_\_'

```
1 import torch.nn as nn
2 class ResidualConnection(nn.Module):
3     # Implements Pre-Norm:  $X_{\text{final}} = X + \text{SubLayer}(\text{LN}(X))$ 
4     def __init__(self, sublayer,
5                 d_model, dropout):
6         super().__init__()
7         self.sublayer = sublayer
8         self.norm = nn.LayerNorm(d_model)
9         self.dropout = nn.Dropout(dropout)
10
11 # --- Example (in your
12 # --- TransformerLayer __init__)
13 # self.mha = MultiHeadAttention(...)
14 # self.ffn = FeedForward(...)
15
16 # self.mha_res = ResidualConnection(
17 #     self.mha, d_model, dropout
18 # )
19 # self.ffn_res = ResidualConnection(
20 #     self.ffn, d_model, dropout
21 # )
```

# PyTorch Code: Add & Norm (Pre-Norm)

## Part 2: 'forward'

```
1  def forward(self, x):
2      # x shape: [B, T, d_model]
3
4      # 1. Normalize first (Pre-Norm)
5      x_norm = self.norm(x)
6
7      # 2. Pass through sublayer (MHA/FFN)
8      sub_output = self.sublayer(x_norm)
9
10     # 3. Apply dropout
11     sub_output = self.dropout(sub_output)
12
13     # 4. Add residual connection
14     return x + sub_output
15
16 # --- Example (in your
17 # --- TransformerLayer forward)
18 # x = ...
19 # x = self.mha_res(x)
20 # x = self.ffn_res(x)
21 # return x
```

# Putting It All Together: A Transformer “Block”

- We have now seen all the “LEGO bricks” (components) needed to build a Transformer layer.
- A single “Transformer Block” (or “Layer”) is just a stack of these components.

## Flow of one Transformer Block

- 1 **Input:**  $\mathbf{X}_{in}$  (from previous layer)
- 2 **Sub-layer 1 (MHA):**
  - $\mathbf{X}_{norm1} = \text{LayerNorm}(\mathbf{X}_{in})$
  - $\mathbf{X}_{mha} = \text{MultiHeadAttention}(\mathbf{X}_{norm1})$
  - $\mathbf{X}_{mid} = \mathbf{X}_{in} + \mathbf{X}_{mha}$  (1st Residual Connection)

# Putting It All Together: A Transformer “Block”

- We have now seen all the “LEGO bricks” (components) needed to build a Transformer layer.
- A single “Transformer Block” (or “Layer”) is just a stack of these components.

## Flow of one Transformer Block

- 1 **Input:**  $\mathbf{X}_{in}$  (from previous layer)
- 2 **Sub-layer 1 (MHA):**
  - $\mathbf{X}_{norm1} = \text{LayerNorm}(\mathbf{X}_{in})$
  - $\mathbf{X}_{mha} = \text{MultiHeadAttention}(\mathbf{X}_{norm1})$
  - $\mathbf{X}_{mid} = \mathbf{X}_{in} + \mathbf{X}_{mha}$  (1st Residual Connection)
- 3 **Sub-layer 2 (FFN):**
  - $\mathbf{X}_{norm2} = \text{LayerNorm}(\mathbf{X}_{mid})$
  - $\mathbf{X}_{ffn} = \text{FeedForward}(\mathbf{X}_{norm2})$
  - $\mathbf{X}_{out} = \mathbf{X}_{mid} + \mathbf{X}_{ffn}$  (2nd Residual Connection)

# Putting It All Together: A Transformer “Block”

- We have now seen all the “LEGO bricks” (components) needed to build a Transformer layer.
- A single “Transformer Block” (or “Layer”) is just a stack of these components.

## Flow of one Transformer Block

- 1 **Input:**  $\mathbf{X}_{in}$  (from previous layer)
- 2 **Sub-layer 1 (MHA):**
  - $\mathbf{X}_{norm1} = \text{LayerNorm}(\mathbf{X}_{in})$
  - $\mathbf{X}_{mha} = \text{MultiHeadAttention}(\mathbf{X}_{norm1})$
  - $\mathbf{X}_{mid} = \mathbf{X}_{in} + \mathbf{X}_{mha}$  (1st Residual Connection)
- 3 **Sub-layer 2 (FFN):**
  - $\mathbf{X}_{norm2} = \text{LayerNorm}(\mathbf{X}_{mid})$
  - $\mathbf{X}_{ffn} = \text{FeedForward}(\mathbf{X}_{norm2})$
  - $\mathbf{X}_{out} = \mathbf{X}_{mid} + \mathbf{X}_{ffn}$  (2nd Residual Connection)
- 4 **Output:**  $\mathbf{X}_{out}$  (to next layer)

- This **entire block** is then stacked  $N$  times (e.g.,  $N = 12$ ) to build the model.

# PyTorch Code: Transformer Block (1/2: '\_\_init\_\_')

## Conceptual PyTorch 'TransformerBlock'

```
1 class TransformerBlock(nn.Module):
2     """
3     Implements one full Transformer Block (MHA -> Add&Norm -> FFN ->
4     Add&Norm) using the Pre-Norm style.
5     """
6     def __init__(self, d_model, h, d_ff, dropout_p):
7         super().__init__()
8         # 1. Multi-Head Attention sub-layer
9         self.mha = MultiHeadAttention(d_model, h)
10        # 2. Feed-Forward sub-layer
11        self.ffn = FeedForward(d_model, d_ff)
12        # 3. Two ResidualConnection wrappers
13        self.res_conn_1 = ResidualConnection(
14            sublayer_module = self.mha,
15            d_model = d_model,
16            dropout_p = dropout_p
17        )
18        self.res_conn_2 = ResidualConnection(
19            sublayer_module = self.ffn,
20            d_model = d_model,
21            dropout_p = dropout_p
22        )
```

# PyTorch Code: Transformer Block (2/2: 'forward')

## Conceptual PyTorch 'TransformerBlock'

```
1  def forward(self, x, mask=None):
2      # x shape: [B, T, d_model]
3      # 1. Apply MHA block:
4      # X_mid = X_in + MHA(LN(X_in))
5      # We must pass the mask to MHA
6      x = self.res_conn_1(x, mask=mask)
7      # 2. Apply FFN block:
8      # X_out = X_mid + FFN(LN(X_mid))
9      # No mask needed for FFN
10     x = self.res_conn_2(x)
11     return x
12
13 # --- Full Model (Conceptual) ---
14 # self.embedding = ...
15 # self.ropes = ...
16 # self.transformer_blocks = nn.ModuleList([
17 #     TransformerBlock(d_model, h, d_ff, dropout)
18 #     for _ in range(N_LAYERS)])
19 # x = self.embedding(input_ids)
20 # for block in self.transformer_blocks:
21 #     x = block(x, mask=my_mask)
```

# So... We Have a 'TransformerBlock', What's next?

- We have successfully built one complete, generic 'TransformerBlock' from its core components (MHA, FFN, Add&Norm).
  - A full Transformer model is simply a **stack** of these blocks. But what is the purpose of this stack?

# So... We Have a 'TransformerBlock', What's next?

- We have successfully built one complete, generic 'TransformerBlock' from its core components (MHA, FFN, Add&Norm).
  - A full Transformer model is simply a **stack** of these blocks. But what is the purpose of this stack?
- The original Transformer was designed for **sequence-to-sequence** tasks (like translation).
- To do this, it proposed an architecture with **two distinct stacks**, each using a specialized version of our block:
  - An **Encoder Stack**: to “understand” the input sequence.
  - A **Decoder Stack**: to “generate” the output sequence.

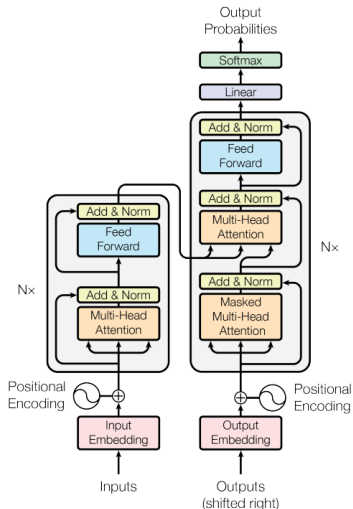
# Encoder vs. Decoder Stacks

- **Encoder Stack (Left):**

- Consists of Multi-Head Attention layers and Feed-Forward layers.
- **Bidirectional:** Each position can see all other positions.

- **Decoder Stack (Right):**

- Consists of **Masked MHA**, **Cross-Attention**, and Feed-Forward layers.
  - **Autoregressive:** Each position can only see **previous** positions.
- **GPT models use **only** the Decoder blocks.**



# Exercise: Conceptual Challenge

## Question: What if we build an Encoder from Decoder Blocks?

- **Scenario:** You are building a model for translation (seq-to-seq).
- You decide to build your **Encoder** stack using 'DecoderBlock' modules (i.e., with a **causal mask**) instead of 'EncoderBlock' modules.
- **Question:** What is the critical flaw in this design? How would it impact the model's performance on a translation task (e.g., translating "I am a student" to Chinese)?
  - **Hint:** Think about what "bidirectionality" means for the Encoder. What information does the Encoder need to create a good representation of the **source** sentence?

# Exercise: Conceptual Challenge

## Question: What if we build an Encoder from Decoder Blocks?

- **Scenario:** You are building a model for translation (seq-to-seq).
- You decide to build your **Encoder** stack using 'DecoderBlock' modules (i.e., with a **causal mask**) instead of 'EncoderBlock' modules.
- **Question:** What is the critical flaw in this design? How would it impact the model's performance on a translation task (e.g., translating "I am a student" to Chinese)?
  - **Hint:** Think about what "bidirectionality" means for the Encoder. What information does the Encoder need to create a good representation of the **source** sentence?

## Answer

- **Flaw:** The Encoder would no longer be bidirectional. When processing the word *am*, it would **only** be able to see *I*. It could **not** see *a student*.
- **Impact:** The contextual representation for each source token would be incomplete. This makes it impossible for the model to understand the full meaning of the source sentence, leading to terrible translation quality.

# Decoder: Masked Multi-Head Attention

- **Purpose:** The “mask” is essential for autoregressive language modeling. It ensures that the prediction for token  $t$  can only depend on previous tokens  $1 \dots t - 1$ .
- **How it works:**
  - It modifies the attention score matrix  $S_i$  before the softmax step.
  - A “mask matrix” (with values of  $-\infty$ ) is added to the scores.

$$S_{i\_masked} = S_i + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

- After softmax, all “future” positions ( $-\infty$ ) will have a probability of 0.

# PyTorch Code: Causal (Look-Ahead) Mask

## Creating and Using the Mask

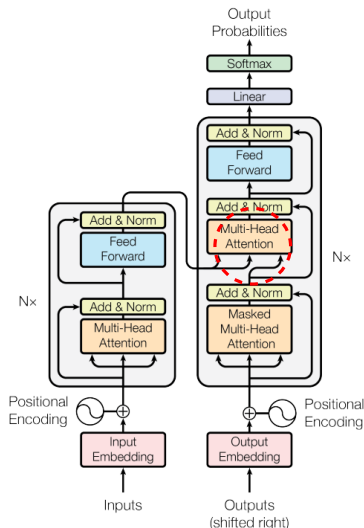
```
1 import torch
2 # 1. Create an upper-triangular matrix of 1s (True)
3 # torch.triu(..., diagonal=1) -> Upper triangle, k=1
4 # [[0., 1., 1., 1., 1.], [0., 0., 1., 1., 1.],
5 #  [0., 0., 0., 1., 1.], [0., 0., 0., 0., 1.],
6 #  [0., 0., 0., 0., 0.]]
7 causal_mask_bool = torch.triu(torch.ones(seq_len, seq_len), diagonal
8                                =1).bool()
9
10 print("Causal Mask (True where we fill):\n", causal_mask_bool)
11
12 # 2. Add Batch and Head dimensions for broadcasting
13 final_mask = causal_mask_bool.unsqueeze(0).unsqueeze(0)
14 print("\nFinal mask shape for SDPA:", final_mask.shape)
15
16 # 3. Test in SDPA function
17 dummy_q = torch.randn(1, 1, seq_len, 64)
18 dummy_k = torch.randn(1, 1, seq_len, 64)
19 dummy_v = torch.randn(1, 1, seq_len, 64)
20 # Pass the mask where True means 'fill'
21 output, weights = scaled_dot_product_attention(dummy_q, dummy_k,
22                                                  dummy_v, mask=final_mask)
```

# Cross-Attention

- **What is it?** A standard MHA module, but with different inputs.
- **Self-Attention (what we've seen):**
  - $Q, K, V$  all come from the **same source** (the previous layer's output).
  - 'output = mha(x, x, x)'
  - "How should tokens in  $X$  attend to **other tokens in  $X$** ?"
- **Cross-Attention (new):**
  - $Q$  comes from the **Decoder**.
  - $K$  and  $V$  come from the **Encoder**.
  - 'output = mha(x\_decoder, encoder\_output, encoder\_output)'
  - "How should Decoder tokens attend to **Encoder tokens**?"

# Cross-Attention

- **What is it?** A standard MHA module, but with different inputs.
- **Self-Attention (what we've seen):**
  - $Q, K, V$  all come from the **same source** (the previous layer's output).
  - 'output = mha(x, x, x)'
  - "How should tokens in  $X$  attend to **other tokens in  $X$** ?"
- **Cross-Attention (new):**
  - $Q$  comes from the **Decoder**.
  - $K$  and  $V$  come from the **Encoder**.
  - 'output = mha(x\_decoder, encoder\_output, encoder\_output)'
  - "How should Decoder tokens attend to **Encoder tokens?**"
- **Purpose:** This is the **only** way the Decoder gets information from the Encoder. It allows the generating (Decoder) stack to "look at" the input (Encoder) stack.



# PyTorch Code: Decoder Block (1/2: '\_\_init\_\_')

## Conceptual PyTorch 'DecoderBlock'

```
1 class DecoderBlock(nn.Module):
2     """
3     Implements one full Decoder Block
4     (Masked MHA -> Add&Norm -> Cross MHA -> Add&Norm -> FFN -> Add&
5     Norm)
6     """
7     def __init__(self, d_model, h, d_ff, dropout_p):
8         super().__init__()
9         # 1. Masked Multi-Head Attention (Self-Attention)
10        self.masked_mha = MultiHeadAttention(d_model, h)
11        # 2. Cross-Attention (MHA)
12        self.cross_mha = MultiHeadAttention(d_model, h)
13        # 3. Feed-Forward
14        self.ffn = FeedForward(d_model, d_ff)
15
16        # 4. Three ResidualConnection wrappers
17        self.res_conn_1 = ResidualConnection(self.masked_mha, d_model,
18        dropout_p)
19        self.res_conn_2 = ResidualConnection(self.cross_mha, d_model,
20        dropout_p)
21        self.res_conn_3 = ResidualConnection(self.ffn, d_model,
22        dropout_p)
```

# PyTorch Code: Decoder Block (2/2: 'forward')

## Conceptual PyTorch 'DecoderBlock'

```
1  def forward(self, x, encoder_output, causal_mask, padding_mask=
    None):
2      # x shape: [B, Target_T, D], encoder_output shape: [B,
        Source_T, D]
3
4      # 1. Apply Masked MHA block (Self-Attention)
5      # Q, K, V all come from x
6      # We must use the causal (look-ahead) mask
7      x = self.res_conn_1(
8          x, x_k=x, x_v=x, mask=causal_mask
9      )
10     # 2. Apply Cross-Attention block
11     # Q comes from x (decoder)
12     # K, V come from encoder_output
13     x = self.res_conn_2(x, x_k=encoder_output, x_v=encoder_output,
        mask=padding_mask # (Masks <pad> tokens in encoder)
14     )
15
16     # 3. Apply FFN block
17     x = self.res_conn_3(x)
18     return x
```

# Decoder: Final Output Distribution (GPT-style)

- After  $N$  decoder layers, we have a final set of token embeddings  $\mathbf{X} \in \mathbb{R}^{t \times d}$ .
- **How to predict the next token?**

- 1 GPT selects only the embedding of the **last** position,  $\mathbf{x}_t$ .
- 2 It computes "logits" by projecting this vector into the vocabulary space using the **initial** embedding matrix  $\mathbf{M}$  (this is called Weight Tying).

$$\mathbf{z}_t = \mathbf{x}_t^\top \mathbf{M} \in \mathbb{R}^{|\mathcal{Y}|}$$

- 3 The logits  $\mathbf{z}_t$  are converted to a probability distribution using the **Softmax** function.

$$P(y|\text{context}) = \frac{\exp(\mathbf{z}_t[y])}{\sum_{y' \in \mathcal{Y}} \exp(\mathbf{z}_t[y'])}$$

# PyTorch Code: Final Output (1/2: '\_\_init\_\_' & Weight Tying)

## Conceptual PyTorch 'GPTModel'

```
1 class GPTModel(nn.Module):
2     def __init__(self, vocab_size, d_model, n_layers, h, ...):
3         super().__init__()
4         # 1. Token Embedding (Matrix M)
5         self.token_embedding = nn.Embedding(vocab_size, d_model)
6         # 2. Stack of N Decoder-Only Blocks
7         # (Note: A GPT block is a DecoderBlock *without* the Cross-
8           Attention sub-layer)
9         self.blocks = nn.ModuleList([
10             GPTDecoderBlock(d_model, h, ...)
11             for _ in range(n_layers)
12         ])
13         # 3. Final Layer Normalization
14         self.final_norm = nn.LayerNorm(d_model)
15         # 4. Final Linear Layer (Projection to Vocab)
16         self.final_linear = nn.Linear(d_model, vocab_size)
17         # 5. !!! WEIGHT TYING !!!
18         # The weights of the embedding layer (M) and the final linear
19         # layer are shared.
20         self.final_linear.weight = self.token_embedding.weight
```

# PyTorch Code: Final Output (2/2: 'forward' Pass)

## Conceptual PyTorch 'GPTModel'

```
1  def forward(self, input_ids, targets=None):
2      # input_ids: [Batch, Seq_Len]
3      # 1. Get Token Embeddings, x shape: [B, T, D]
4      x = self.token_embedding(input_ids)
5      # 2. Pass through all Decoder Blocks
6      for block in self.blocks:
7          x = block(x)
8      # 3. Final Norm
9      x = self.final_norm(x)
10     # 4. Project to Vocab Space (Logits)
11     logits = self.final_linear(x)
12     # logits shape: [B, T, Vocab_Size]
13     # 5. Calculate Loss (During Training)
14     if targets is not None:
15         # Flatten for CrossEntropyLoss
16         # (B, T, V) -> (B*T, V); (B, T) -> (B*T)
17         loss = F.cross_entropy(logits.view(-1, logits.size(-1)),
18                                targets.view(-1))
19     else:
20         loss = None
21     return logits, loss
```

# Full 'Transformer' (1/4: 'Encoder')

## The 'Encoder' Module

```
1  # (We assume EncoderBlock is already defined)
2
3  class Encoder(nn.Module):
4      def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
5          super().__init__()
6          # Create N identical EncoderBlocks
7          self.layers = nn.ModuleList([
8              EncoderBlock(d_model, h, d_ff, dropout_p)
9              for _ in range(n_layers)
10          ])
11          # Final normalization
12          self.norm = nn.LayerNorm(d_model)
13
14      def forward(self, x, mask=None):
15          # x shape: [B, T_src, D]
16          # Pass input through all N layers
17          for layer in self.layers:
18              x = layer(x, mask=mask)
19
20          # Final normalization
21          return self.norm(x)
```

# Full 'Transformer' (2/4: 'Decoder')

## The 'Decoder' Module

```
1  # (We assume DecoderBlock is already defined)
2  class Decoder(nn.Module):
3      def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
4          super().__init__()
5          # Create N identical DecoderBlocks
6          self.layers = nn.ModuleList([
7              DecoderBlock(d_model, h, d_ff, dropout_p)
8              for _ in range(n_layers)
9          ])
10         # Final normalization
11         self.norm = nn.LayerNorm(d_model)
12
13     def forward(self, x, enc_output, causal_mask, padding_mask):
14         # x shape: [B, T_tgt, D]
15         # Pass input through all N layers
16         for layer in self.layers:
17             x = layer(x, encoder_output=enc_output, causal_mask=
18                     causal_mask, padding_mask=padding_mask)
19         # Final normalization
20         return self.norm(x)
```

# Full 'Transformer' (3/4: Final Assembly '\_\_init\_\_')

## The 'Transformer' Model

```
1 class Transformer(nn.Module):
2     def __init__(self, src_vocab_size, tgt_vocab_size, d_model,
3         n_layers, h, d_ff, dropout_p, max_len):
4         super().__init__()
5         # --- Embeddings ---
6         # (We use nn.Embedding for absolute pos,
7         #  but RoPE would be inside MHA)
8         self.src_embedding = nn.Embedding(src_vocab_size, d_model)
9         self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)
10        self.pos_encoding = PositionalEncoding(d_model, dropout_p,
11            max_len)
12
13        # --- Encoder & Decoder Stacks ---
14        self.encoder = Encoder(n_layers, d_model, h, d_ff, dropout_p)
15        self.decoder = Decoder(n_layers, d_model, h, d_ff, dropout_p)
16
17        # --- Final Output Layer ---
18        self.final_linear = nn.Linear(d_model, tgt_vocab_size)
19
20        # --- (Optional) Weight Tying ---
21        # self.final_linear.weight = self.tgt_embedding.weight
```

# Full 'Transformer' (4/4: Final Assembly 'forward')

## The 'Transformer' Model

```
def forward(self, src_ids, tgt_ids, src_mask, tgt_mask):
    # src_ids: [B, T_src]; tgt_ids: [B, T_tgt]
    # src_mask: [B, 1, 1, T_src] (for padding)
    # tgt_mask: [B, 1, T_tgt, T_tgt] (causal + padding)

    # 1. Encode the source sequence
    # src_embed: [B, T_src, D]
    src_embed = self.pos_encoding(self.src_embedding(src_ids))
    # enc_output: [B, T_src, D]
    enc_output = self.encoder(src_embed, mask=src_mask)

    # 2. Decode the target sequence
    # tgt_embed: [B, T_tgt, D]
    tgt_embed = self.pos_encoding(self.tgt_embedding(tgt_ids))
    # dec_output: [B, T_tgt, D]
    dec_output = self.decoder(tgt_embed, enc_output, causal_mask=
                              tgt_mask, padding_mask=src_mask)

    # 3. Final projection to logits
    # logits: [B, T_tgt, tgt_vocab_size]
    logits = self.final_linear(dec_output)
    return logits
```

# Transformer: A Foundational Revolution

- The 2017 “Attention Is All You Need” paper was a paradigm shift, replacing recurrence (RNNs) with self-attention.
- **Why did it win?** It enabled **massive parallelization** and **scalability**.
- It is the **architectural foundation** for nearly all modern AI models:
  - BERT (Encoder-only)
  - DeepSeek, GPT, Qwen, Llama (Decoder-only)
  - T5, BART (Encoder-Decoder)

# Drawbacks of Transformers

## Quadratic Compute Cost ( $\mathcal{O}(N^2)$ )

- Computing all pairs of interactions in self-attention (the  $Q \cdot K^\top$  matrix) grows **quadratically** with sequence length  $N$ .
- Doubling the context length  $\rightarrow$  4x the computation in the attention layer.
- This makes training on very long sequences computationally prohibitive.

## Large Memory Footprint (KV Cache)

- During autoregressive decoding (generation), we must store the  $K$  and  $V$  tensors for **all past tokens**.
- This “KV Cache” grows **linearly** with sequence length ( $\mathcal{O}(N)$ ).
- This is often the memory bottleneck that limits context size during inference, not the model weights.

## Fixed Context Window

- The model is trained on a fixed length (e.g., 2048).
- It cannot **naturally** generalize to sequences longer than what it was trained on (extrapolation).

# Generative Pre-trained Transformer (GPT) Models

# The Rise of Decoder-Only Models

- As we've seen, the Transformer architecture is flexible:
  - **Encoder-Only** (e.g., BERT, for NLU tasks)
  - **Encoder-Decoder** (e.g., T5, for translation)
  - **Decoder-Only** (for generation)
- In the current era of LLMs, one paradigm has become the most dominant and powerful for general-purpose AI: **Decoder-Only**.

# The Rise of Decoder-Only Models

- As we've seen, the Transformer architecture is flexible:
  - **Encoder-Only** (e.g., BERT, for NLU tasks)
  - **Encoder-Decoder** (e.g., T5, for translation)
  - **Decoder-Only** (for generation)
- In the current era of LLMs, one paradigm has become the most dominant and powerful for general-purpose AI: **Decoder-Only**.
- The most famous example is the **GPT series** (Generative Pre-trained Transformer) from OpenAI.
- These models are simply **stacks of Decoder blocks** (like we've seen, but without the cross-attention part) trained on the core task of **Next-Token Prediction**.

# The 3-Stage Training Pipeline

- Modern GPT models (e.g., ChatGPT, GPT-4) are trained in a multi-stage process to align them with human preferences.
  - Step 1: **Pre-training** (General knowledge)
  - Step 2: **Supervised Fine-Tuning (SFT)** (Instruction following)
  - Step 3: **Reinforcement Learning from Human Feedback (RLHF)** (Preference alignment)

# Stage 1: Pre-training

- **Model:** A GPT (Decoder-Only Transformer).
- **Data ( $\mathcal{D}$ ):** Massive, diverse, unlabeled text (e.g., Wikipedia).
- **Task:** Autoregressive Next-Token Prediction.
- **Objective:** Minimize the negative log-likelihood (NLL) of the data.

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(y_1, \dots, y_T) \in \mathcal{D}} \sum_{t=1}^T \log P_{\theta}(y_t | y^{t-1})$$

# Stage 1: Pre-training

- **Model:** A GPT (Decoder-Only Transformer).
- **Data ( $\mathcal{D}$ ):** Massive, diverse, unlabeled text (e.g., Wikipedia).
- **Task:** Autoregressive Next-Token Prediction.
- **Objective:** Minimize the negative log-likelihood (NLL) of the data.

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(y_1, \dots, y_T) \in \mathcal{D}} \sum_{t=1}^T \log P_{\theta}(y_t | y^{t-1})$$

- **Result:** A **Base Model** that is a very powerful text completer but may not follow instructions well.

# SFT & RLHF

## Step 1

Collect demonstration data and train a supervised policy.

A prompt is sample from our prompt dataset.

 Explain reinforcement learning to a 6 year old.

A labeler demonstrates the desired output behavior.

 We give treats and punishments to teach...

This data is used to fine-tune GPT-3.5 with supervised learning.

 SFT  


## Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

 Explain reinforcement learning to a 6 year old.

 A In reinforcement learning, the agent is...

 B Explain rewards...

 C In machine learning...

 D We give treats and punishments to teach...

A labeler ranks the outputs from best to worst.

 D > C > A > B

This data is used to train our reward model.

 RM  
D > C > A > B

## Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.

 Write a story about otters.

The PPO model is initialized from the supervised policy.

 PPO

The policy generates an output.

Once upon a time...

The reward model calculates a reward for the output.

 RM

The reward is used to update the policy using PPO.

$r_k$

## Stage 2: Supervised Fine-Tuning (SFT)

- **Goal:** Teach the base model to act as an assistant by following instructions.
- **Data ( $\mathcal{D}_{fine-tune}$ ):** A smaller, high-quality dataset of (prompt  $x$ , human-annotated answer  $y$ ) pairs.
- **Objective:** Minimize the NLL loss, starting from the pre-trained model's weights  $\theta$ .

$$\mathcal{L}_{fine-tune}(\theta) = -\frac{1}{|\mathcal{D}_{fine-tune}|} \sum_{(x,y) \in \mathcal{D}_{fine-tune}} \log P_{\theta}(y|x)$$

## Stage 2: Supervised Fine-Tuning (SFT)

- **Goal:** Teach the base model to act as an assistant by following instructions.
- **Data ( $\mathcal{D}_{fine-tune}$ ):** A smaller, high-quality dataset of (prompt  $x$ , human-annotated answer  $y$ ) pairs.
- **Objective:** Minimize the NLL loss, starting from the pre-trained model's weights  $\theta$ .

$$\mathcal{L}_{fine-tune}(\theta) = -\frac{1}{|\mathcal{D}_{fine-tune}|} \sum_{(x,y) \in \mathcal{D}_{fine-tune}} \log P_{\theta}(y|x)$$

- **Result:** An **SFT Model** that is good at instruction-following.

# Stage 3 (RLHF): Training the Reward Model (RM)

- **Goal:** Create a model,  $r_{\theta}(x, y)$ , that outputs a scalar score for how “good” an answer  $y$  is for a prompt  $x$ .
- **Data Collection:**
  - 1 Take a prompt  $x$ .
  - 2 Use the SFT model to generate  $K$  different answers ( $y_1 \dots y_K$ ).
  - 3 Have human labelers **rank** these  $K$  answers from best to worst (e.g.,  $y_w > y_l$ ).
- **Dataset:** This creates a **Human Preference Dataset ( $\mathcal{D}$ )** of ranked pairs  $(x, y_w, y_l)$

# Stage 3 (RLHF): Training the Reward Model (RM)

- **Goal:** Create a model,  $r_\theta(x, y)$ , that outputs a scalar score for how “good” an answer  $y$  is for a prompt  $x$ .
- **Data Collection:**
  - 1 Take a prompt  $x$ .
  - 2 Use the SFT model to generate  $K$  different answers ( $y_1 \dots y_K$ ).
  - 3 Have human labelers **rank** these  $K$  answers from best to worst (e.g.,  $y_w > y_l$ ).
- **Dataset:** This creates a **Human Preference Dataset ( $\mathcal{D}$ )** of ranked pairs  $(x, y_w, y_l)$
- **Objective:** Train the RM  $r_\theta$  to predict the human ranking. The loss function maximizes the score gap between preferred ( $y_w$ ) and rejected ( $y_l$ ) answers:

$$\text{loss}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$$

# Stage 3 (RLHF): Optimizing with PPO

- **Goal:** Use the trained RM as the “reward” signal to train the SFT model (now called the “policy”  $\pi_\phi$ ).
  - **Algorithm:** Proximal Policy Optimization (PPO).
- **PPO Objective (Maximize):**

$$obj(\phi) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\pi_\phi}} [r_\theta(x,y) - \beta \log(\frac{\pi_\phi(y|x)}{\pi_{SFT}(y|x)})] - \gamma \mathbb{E}_{x \sim \mathcal{D}_{pretrain}} [\log(\pi_\phi(x))]$$

# Stage 3 (RLHF): Optimizing with PPO

- **PPO Objective (Maximize):**

$$obj(\phi) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\pi_\phi}} [r_\theta(x, y) - \beta \log(\frac{\pi_\phi(y|x)}{\pi^{SFT}(y|x)})] - \gamma \mathbb{E}_{x \sim \mathcal{D}_{pretrain}} [\log(\pi_\phi(x))]$$

- **Data Source 1 (Policy Data  $\mathcal{D}_{\pi_\phi}$ ):**

- Prompts ( $x$ ) are sampled from a prompt dataset.
- The current policy ( $\pi_\phi$ ) generates responses ( $y$ ).
- The RM scores this ( $x, y$ ) pair to get the **reward**  $r_\theta(x, y)$ .
- $\beta \cdot \log(\dots)$ : **KL Penalty**. Prevents  $\pi_\phi$  from straying too far from  $\pi^{SFT}$ .

- **Data Source 2 (Pre-training Data  $\mathcal{D}_{pretrain}$ ):**

- The original pre-training corpus is used to compute the **entropy regularization** term, encouraging exploration and preventing the model from “forgetting” its language capabilities.

# Beyond Closed-Source: The Open-Source Ecosystem

- The 3-stage process (Pre-training, SFT, RLHF) is how powerful models like OpenAI's **ChatGPT** and **GPT-4/5** are built.
- However, these state-of-the-art models are “**closed-source**” or available only via paid APIs. We cannot inspect their weights or architecture.
- This has sparked a massive **open-source movement** to build powerful, transparent, and accessible models.

# Beyond Closed-Source: The Open-Source Ecosystem

- The 3-stage process (Pre-training, SFT, RLHF) is how powerful models like OpenAI's **ChatGPT** and **GPT-4/5** are built.
- However, these state-of-the-art models are “**closed-source**” or available only via paid APIs. We cannot inspect their weights or architecture.
- This has sparked a massive **open-source movement** to build powerful, transparent, and accessible models.
- Key **Open-Source** Models:
  - **DeepSeek** (from DeepSeek AI)
  - **Qwen** (from Alibaba)
  - **Llama** (from Meta)
  - **Mistral** (from Mistral AI)
  - ...
- How can we use these models?

# Hugging Face 'transformers': The Gateway to LLMs

- The **Hugging Face 'transformers'** library is the standard tool for working with open-source models.
- It provides a simple, unified API to download, load, and use thousands of pre-trained models.

# Hugging Face 'transformers': The Gateway to LLMs

- The **Hugging Face 'transformers'** library is the standard tool for working with open-source models.
- It provides a simple, unified API to download, load, and use thousands of pre-trained models.

## The 'pipeline' (Easy Mode)

```
1 from transformers import pipeline
2 # 1. Load a pre-built pipeline
3 pipe = pipeline(
4     "text-generation",
5     model="Qwen/Qwen2-0.5B-Instruct"
6 )
7 # 2. Use it!
8 messages = [
9     {"role": "user", "content": "Write a hello world in Python"}
10 ]
11 output = pipe(messages, max_new_tokens=20)
12 print(output[0]['generated_text'][-1]['content'])
13 # --- Output ---
14 # def hello_world():
15 #     print("Hello, world!")
```

# The 'AutoModel' Workflow (e.g., Qwen)

- The "standard" 'AutoModel' workflow gives you more control over tokenization and generation.

## Example: Load 'Qwen' Manually

```
1 from transformers import AutoTokenizer, AutoModelForCausalLM
2 model_name = "Qwen/Qwen2-0.5B-Instruct"
3 # 1. Load Tokenizer
4 tokenizer = AutoTokenizer.from_pretrained(model_name)
5 # 2. Load Model
6 model = AutoModelForCausalLM.from_pretrained(model_name)
7 # (Add ".to("cuda") if you have a GPU)
8 # 3. Format & Tokenize Prompt
9 messages = [
10     {"role": "user",
11      "content": "Tell me a short joke"}
12 ]
13 text = tokenizer.apply_chat_template(messages, tokenize=False)
14 inputs = tokenizer(text, return_tensors="pt")
15 # 4. Generate!
16 outputs = model.generate(**inputs, max_new_tokens=50)
17 # 5. Decode and Print
18 result = tokenizer.decode(outputs[0], skip_special_tokens=True)
19 print(result)
```

## “Before”: Greedy Search (Default)

- ‘model.generate(...)’ by default uses **Greedy Search**.
- It always picks the **single most probable** next token.
- It is fast, deterministic, but often “boring” and repetitive.

```
1 # This is greedy search
2 # (do_sample=False is default)
3 outputs = model.generate(
4     **inputs,
5     max_new_tokens=50
6 )
7 print(tokenizer.decode(outputs[0]))
```

# Beam Search

- **Beam Search** explores  $k$  (e.g., 'num\_beams=4') possible paths at each step.
- It finds the **entire sequence** with the highest total probability.
- It's often more coherent than Greedy, but still deterministic and can be repetitive.

## Use Beam Search & Fix Repetition

```
1 # Task 1: Use 4 beams to find a better joke
2 beam_outputs = model.generate(
3     **inputs,
4     max_new_tokens=50,
5     num_beams=4,
6     early_stopping=True # Stop when all beams finish
7 )
8 print(tokenizer.decode(bean_outputs[0], ...)) # Often better!
9 # Task 2: What if it repeats? "I I I I..."
10 # Use repetition_penalty to discourage repeated tokens
11 beam_outputs_fixed = model.generate(
12     **inputs,
13     max_new_tokens=50,
14     num_beams=4,
15     early_stopping=True,
16     repetition_penalty=1.2 # > 1.0 penalizes repeats
17 )
18 print(tokenizer.decode(bean_outputs_fixed[0], ...))
```

# Practice: Creative Sampling (to be submitted)

- **Sampling** introduces randomness for more diverse, creative results.
- **Top-K Sampling** ('top\_k=50'): Randomly picks from the 50 most likely tokens.
- **Top-p (Nucleus) Sampling** ('top\_p=0.9'): Randomly picks from the smallest set of tokens whose cumulative probability exceeds 90%.  
This is generally preferred.
- **Your Task:** Compare 'top\_k' vs. 'top\_p'.

# Scaling Laws & Future Challenges

# The Scaling Law

- **Observation:** It has been observed that the performance of a large language model (LLM) scales predictably with its **model size** (number of parameters).
- **Implication:** This hypothesis has driven the trend toward building larger and larger models, as “bigger has been better”.

# The Scaling Law

- **Observation:** It has been observed that the performance of a large language model (LLM) scales predictably with its **model size** (number of parameters).
- **Implication:** This hypothesis has driven the trend toward building larger and larger models, as “bigger has been better”.
- Except model size, some other research also shows that performance scales with **dataset size**, and **compute**. And these must be scaled in balance.

# Active Research & Future Challenges

- **Understanding Capabilities:** How do architecture, data, and training **really** impact reasoning, generalization, and interpretability?
- **Undesired Behaviors:** Active research to understand and mitigate harmful outputs, especially **hallucination** (generating plausible but false information).
- **Alignment:** How do we robustly **align** models to human values and ensure they are helpful, honest, and harmless?
- **Future Work:** Uncover the **fundamental principles** to create more capable, efficient, and reliable models.
- **This remains an active area of research. If you are interested, feel free to reach out to me anytime!**

- **LMs:** are fundamentally models of **next-token prediction**.
- **Transformer:** a powerful, parallelizable architecture built on **self-attention**.
- **GPT:** a **decoder-only** Transformer.
- **Training Pipeline:**
  - 1 **Pre-training** (general knowledge)
  - 2 **SFT** (instruction following)
  - 3 **RLHF** (preference alignment)
- This is a rapidly evolving field, with many open questions.

# Thank You!